

Call For Participants: QDA-VC competition at AIIDE 2026

Fiona Shyne

March 2, 2026

1 Introduction

We are opening a call for participants to submit algorithms within a novel space: Quality Diversity Algorithms with Variable Constraints (QDA-VC). For many tasks, particularly within PCG and games, the end user desires a diverse suite of high-quality solutions. However, the user likely has a variety of desires for the solutions and these desires are likely to change over time as the user interacts with the system. Thus, there is a need for algorithms that can adapt to the changing desires of an end-user while still producing solutions that are diverse and high-quality. In this system, we represent the desires of a user as a constraint on the solution.

Participants can rely on the QDA-VC benchmark to develop their algorithms. This benchmark provides three problem spaces, four procedural personas, and 4 baseline algorithms. Competing algorithms should preform well across all procedural personas and problem spaces - including a secret problem space that will only be revealed after the competition has ended. The baseline algorithms represent the first attempts at converting existing genetic algorithm techniques to this space. They can be used as inspiration and to test against. The goal of each algorithm is to take in the problem space, static constraints, and current variable constraints and return a set of feasible solutions divided into several bins based on a diversity metric. Algorithms are primarily evaluated through the QD-Score at the end of each generation, which is the sum of the top fitness in each bin. See section 7 for more details about the QDA-VC environment.

2 Reasons to participate

- Test and develop your personal algorithm development skills
- Contribute to the research community and efforts towards interactive genetic algorithms
- Be recognized at the AIIDE-26 conference

- (Optional) The winner will be provided a coffee of their choosing at the AIIDE-26 conference and a chance to have a one-on-one conversation with the organizer of this competition (and developer of the QDA-VC benchmark)

3 Important Dates

- Call for competition posted March 31st
- Submissions open August 1st
- Submissions due September 8th
- Results announced November 13th

4 Rules

1. Anyone over the age of 18 is eligible to participate, regardless of their participation in the AIIDE 2026 conference.
2. All submissions must be self-sufficient and not make use of any external APIs. While most Python libraries can be used, expect your code to be run without internet access.
3. Each generation can last no longer than 30 seconds.
4. All participants must acknowledge that their code and description may be made public after the competition (with proper citation). The goal of this competition to encourage innovation, and as such the technical developments will be shared with the research community.
5. Submissions can be done individually or in teams. However, each participant can be involved in up to one submission.

5 Submission

Submissions must include both a code-base and a textual description of their algorithm. This can be sent in either a .zip file or through a public GitHub repository (where textual description is included in the readme or as a pdf).

Submissions can be email to the organizer, Fiona Shyne (shyne.f@northeastern.edu).

5.1 Code

All submissions for this competition must be written in Python 3.10.12 (this is the version the environment is written in).

The code must include a main.py file that contains a class named Algorithm that is a child of the VariableConstraintAlgorithm class. The Algorithm class

must overwrite the “one-generation” method with the code to run one generation of the algorithm. They can additionally overwrite the ”set_up” method to initialize any data structures before the first generation. The submission can include any additional python files, however they must abide by the rules of the competition. You must include a requirements.txt file with all packages used in the code. This must be able to be able to execute with a “pip install -r requirements.txt” on a new virtual environment. Please test that this works, as often techniques like “pip freeze” will create requirement files that crash. Not being able to run your code can cause you to be disqualified from the competition.

5.1.1 VariableConstraintAlgorithm class

As a child of the VariableConstraintAlgorithm class, submission classes will have access to the following variables/functions:

- Mutation rate: value of 0-1 representing rate of mutation. This value is chosen per problem space
- Population size: how many individuals you can create per generation
- Archive size: how many individuals can be saved in memory at any one given time
- Variable Constraints: the current set of variable constraints
- Problem Space: the problem space will provide all necessary information about the environment, given below.
 - Static constraints: the constraints of the environment that remain true at all times
 - Number of bins: the number of bins to place individuals in
 - Diversity: Given an individual, which bin should they be placed in
 - Random Individual: returns a random individual in the environment
 - Mutate: Given an individual and mutation rate, return a mutated version of the individual
 - Cross-over: Given two individuals, return two children then represent blends of the original individuals
 - Fitness: given an individual, return it’s fitness

5.1.2 set_up method

This method will be run at the beginning of each trial. This should set/re-set all variables so that the algorithm can be run properly for the first generation. This will likely include initializing the population(s) with individuals, but this is limited by population size and archive size. The set_up function can be thought of as the 0th generation of the algorithm

5.1.3 one_generation

This is the method that runs a single generation of the algorithm.

This method has the following input:

- `cons_change`: true if the constraints have been changed since last generation, false otherwise

It also has access to all the variables available to the parent class, as described above.

The method must return a single output, a list of lists containing the generated feasible individuals. Each index represents one cell according to the diversity function of the problem space. Within each cell, there should be a list of all the individuals in the population(s) that: pass all static constraints, pass all variable constraints, are placed in that cell according to the diversity function. The cell should be an empty list if no individuals meet that criteria. Each individual should be placed in the cell list as a tuple, with the first value being its fitness and the second being the individual itself. Each cell should be sorted from highest to lowest fitness.

5.2 Textual Description

In addition to the code, participants must submit a short description of how the algorithm works. This should be under 2 pages (single column), and give a high-level overview of the implementation. This can be writing in a combination of prose and pseudo code (however, please don't just include pseudo code). See subsection 7.3.4 as an example of a textual description.

In addition to the algorithm description please provide the following information:

- Author(s) — These do not have to be legal names, but how you would like to be cited in publications
- Name of the algorithm
- Any previous work/algorithms that served as inspiration for your implementation

6 Evaluation

Each submission will be run across 16 conditions, from the combination of 4 problem spaces and 4 procedural personas. As mentioned in subsection 7.1, 3 of the problem spaces are available in the codebase to test with and 1 space will be kept secret until after the competition. The procedural personas are used to simulate users, and are described in subsection 7.2. In addition to competition submissions, 4 baseline algorithms will be included as described in subsection 7.3. Within each condition, each algorithm will be run for 20 trials with 300 generations, a population size of 200, and maximum archive

memory of 500. The trials will be run on the competition computers which will enforce: maximum run time per generation of 30 second, all individuals are feasible by both the static and variable constraints, the returned fitness values are correct. If any of these are violated, the QD-score for that generation will be 0. Population size and archive memory will be enforced through manual evaluation of the code.

QD-score (sum of top fitness across diversity conditions), will be first averaged across generations and then averaged across trials. After all trials are run, QD-scores from each condition will be scored between 0 and 1 proportional to the worst average QD-score from that condition to the best average QD-score. The final score for each algorithm will be calculated by summing each of its condition scores. This means the maximum score possible is 16, for an algorithm with the top QD-score across all conditions.

In addition to the absolute rankings, we will present on the rankings for each problem space and each persona.

7 Environment

In the following sections we describe the environment of the QDA-VC benchmark. Note that this code is given as references, but none of what is described should be modified for submission. The only aspect competitors should modify is creating Python files to represent their algorithm.

7.1 Problem Spaces

The problem space defines the environment that solutions evolve in. The QDA-VC Benchmark requires a specific problem space where the goal is to generate a diversity of high-quality individuals that satisfy both a set of static constraints, and a set of variable constraints that can change throughout evolution. We consider the case where changes are done by some end-user, but this same toolkit could be applied to other dynamic environments. Unlike similar quality-diversity algorithms (e.g. MAP-Elites), the problem spaces only consider one behavioral metric — although multiple metrics can be mapped into the 1D space.

Every problem space requires the following elements to be defined:

- A set I of possible individuals in the population.
- A set S of static constraints that do not change, and a set V of possible variable constraints, which can be modified by the environment (e.g. user). All possible constraints are then $C = S \cup V$.
- A function $J : \emptyset \rightarrow I$ that returns a random individual in the current environment.
- A constraint evaluation function $E : I \times C \rightarrow B$ that returns *true* if the individual satisfies the constraint, *false* otherwise.

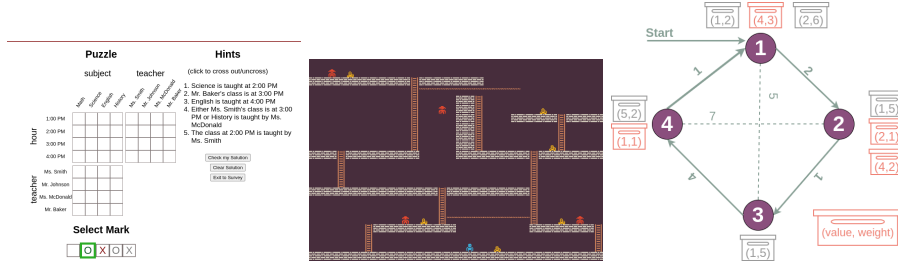


Figure 1: A visual example for each problem space. From left to right: Logic Grid Puzzles Shyne et al. (2024), Lode Runner Khalifa et al. (2025), and the Traveling Thief Problem

- A function that takes a subset of possible constraints $O : P(C) \rightarrow B$ that returns *true* if the given constraints are inherently contradictory, *false* otherwise.¹
- A value b for the number of bins.
- A diversity function $D : I \rightarrow \{0, 1, \dots, b-1\}$, indicating how the individual varies on a behavioral characteristic.
- A fitness function $F : I \rightarrow R$, that must be written as a maximization function.
- A mutation function $M : I \rightarrow I$.
- A cross-over function $X : I \times I \rightarrow I$.
- A function $R : \emptyset \rightarrow V$ that returns a random variable constraint.
- A function $R' : I \rightarrow V$ that returns a random variable constraints that the individual satisfies.

Algorithms will be tested on 4 problem spaces. Three of which are provided with full implementation details, and one which will be a secret up until the competition date. The details of the three provided problem spaces are given below.

7.1.1 Logic Grid Puzzles

The first test problem space is logic grid puzzles. This builds on previous work Shyne et al. (2024), which defined a constrained quality-diversity algorithm for generating logic grid puzzles — a type of puzzle that involves deducing relationship between different entities using a set of natural language hints (see

¹Note this does not require that an individual exists that satisfies all constraints, just if the constraints are not explicitly contradictory. This is used to prevent personas from providing conflicting constraints.

Figure 1 as an example). Later work Shyne et al. (2025) integrated this algorithm into an user interface which allowed users to filter puzzles based on their hints. However, this filtering process was separate from the algorithm so there was no way to optimize to include all user provided constraints. We modified this environment to fit into the current benchmark.

Representation

A puzzle is represented as a list of hints. Individuals are initialized by randomly generating between 1 and 5 hints using the grammar provided in previous work. Individuals are mutated by randomly adding or randomly removing hints from the hint list. Cross-over is done by randomly distributing hints between the two children.

Fitness (F) The fitness of a puzzle is the number of hints in the list subtracted from 15. This was done as it was found that smaller hint sizes tend to reduce redundancy.

Diversity (D)

The diversity is determined by the estimated difficulty, which is based on how many steps an automated solver takes to complete the puzzle. The difficulty values tend to be between 1 and 8, so this problem space had 8 bins — with all puzzles of difficulty greater than or equal to 8 being placed in the last bin.

Static Constraints (S)

Static constraints determine if the hint list represents a valid and solvable puzzle. The solver will attempt to use the hints to solve the puzzle and return a partially or fully complete puzzle state. The puzzle is determined solvable if: each sub-grid is complete (all marks are filled in), each sub-grid is valid (one true mark per row and column), and there are not contradictions in the grid (e.g if $a = b$ and $b = c$, a must equal c).

Variable Constraints (V)

Variable constraints are defined as true if a given hint is included in the individual’s hint list. Random constraints are created by creating a random hint (provided by previous work), or randomly choosing a hint within an existing individual. A set of variable constraints are determined to be contradictory if the solver finds a contradiction when applying all the given hints to a blank puzzle state.

7.1.2 Lode Runner

The Procedural Content Generation Benchmark Khalifa et al. (2025), defines a series of problem spaces for generating content for games. Each problem space has three measures: quality, diversity, and controllability. These measures are similar to the metrics we require in the QDA-VC benchmark, but they cannot be directly mapped. For example “quality” measures both constraints for feasibility (e.g. playability) and optimization criteria. Further, “diversity” provides a measure of how similar two individuals are, however does not easily provide a way to place individuals into a bin based on diversity. Lastly, controllability describes an ideal condition (e.g. number of enemies) and how close an individual is to meeting this condition. Controllability could either be applied to the

diversity function (D) or be used to make variable constraints.

Due to the differences in problem landscapes of the PCG Benchmark and the QDA-VC Benchmark, we selected one of the problem spaces from the PCG Benchmark to manually modify and add to our current benchmark. The problem space we chose is the Lode Runner problem space — a problem space for creating levels for the game Lode Runner. The PCG Benchmark provided the three measures described above (quality, diversity, and controllability), along with additional information that we were able to use. Using the original measures as a guideline we defined the Lode Runner VC-GA problems with the following metrics ².

Representation Each individual level is represented as a 2D array, where each element is a tile in the game. Levels are randomly generated by sampling from the set of tiles, mutated by interpolating between an individual and a randomly generated level (where each cell has a 10% chance of being randomized), and crossed over by randomly interpolating between the two parents (where each cell has an equal chance of being selected from either parent).

Fitness (F)

The horizontal and vertical groups of ladder, ropes, and bricks should fall in the same distribution as the original Lode Runner. This metric is added to encourage the system produces nice-looking levels and not noisy-looking ones.

Diversity (D)

Number of enemies, with a range of two enemies in each bin (e.g. the first bin contains 0-1 enemies, the second 2-3) with a total of 10 bins. Any individual with at least 18 enemies is placed in the last bin.

Static Constraints (S) For each of the following constraints, we added additional constraints to guide the algorithm towards success — as feasibility is measured as number of satisfied constraints in this space. For example, while there should be only one player, we also added that there has to be $\leq 2, 3, 5, 10, 20, 30, 40$ players.

- There has to be one player.
- The player should be able to explore at least 20% of the level.
- The player is able to collect 90% of the gold from the starting location.
- The player is able to reach 75% of the tile surfaces that can be used. This means if there is a thick floor of bricks only the top bricks need the player to be able to move on them.

Variable Constraints (V)

A user can define a variable constraint for wanting a minimum or maximum number of the following items: ropes, ladders, and gold. Constraints are considered contradictory if there are more than one of a type (e.g. two minimum constraints for ropes), or if a minimum and maximum constraint create

²Full details can be found in the benchmark https://github.com/amidos2006/pcg_benchmark

an invalid range of values (e.g. a minimum of 10 ropes and a maximum of 8 ropes).

7.1.3 Traveling Thief Problem

The Traveling Thief Problem (TTP), combines two classic optimization problems: the Traveling Salesman and the Knapsack. In this version of the problem you roleplay a thief traveling to different cities and collecting items at each city, trying to collect as much profit as possible while not being slowed down by the weight of your collected items. For each instance of this problem you are given 1) a set of n nodes (cities), each with an edge to every other node labeled with the distance needed to travel to that node (d_{x_i, x_j}), and 2) a set of m items, each with a weight, a value (p_i), and an assigned node. A solution to the problem is both a tour (a path that visits every node and returns to the starting node), and the set of items that are picked up at each node. The goal is to maximize the sum of the value of the collected items while being penalized for the sum of the weight of the collected items for each unit of travel you take on the tour. This is calculated using the formula below.

$$Z(\Pi, P) = \sum_{i=1}^m p_i y_i - R \left(\frac{d_{x_n, x_1}}{V_{max} - v w_{x_n}} + \sum_{i=1}^{n-1} \frac{d_{x_i, x_{i+1}}}{V_{max} - v w_i} \right)$$

Where $\Pi = (x_1, x_2, \dots, x_n)$ is a tour, $P = (y_1, y_2, \dots, y_m)$ is a packing plan (where y_i is 1 when the i th item is packed and 0 otherwise), R is the renting rate, V_{min} and V_{max} are giving in the problem description as the minimum and maximum speed of the thief, W is the maximum weight the knapsack can hold, w_i is the weight of the knapsack after city i is visited, and $v = \frac{V_{max} - V_{min}}{W}$.

Polyakovskiy et al. (2014) created a set of benchmark instances of the TTP. These benchmarks combined several pre-existing Traveling Salesman problems with Knapsack problems where the association between the value and profit was either *uncorrelated* (uniformly distributed random integers for each), *uncorrelated with similar weights* (like uncorrelated but with a much smaller range for weights), and *bounded strongly correlated* (weight and profits are highly related) items. Within our problem space, we selected 6 instances of the TTP instances in the QDA-VC benchmark, each instance either having 50 or 150 cities and having *uncorrelated*, *uncorrelated with similar weights*, or *bounded strongly correlated* items. Nikfarjam et al. (2024) described how quality-diversity algorithms can be used for the traveling thief problem where the behavior metrics are the length of the tour and the total profit. We used this as inspiration for adapting the TTP into this benchmark toolkit.

Representation

Each solution to the TTP in our problem space is two arrays: the first array representing the sequence of cities that are visited, and the second array having an element for each item with each element being 0 if the item is left or 1 if it is taken.

Solutions are initialized by randomly generating a sequence of cities and randomly assigning 0 or 1 to each item. Mutation is done by randomly swapping cities in the tour, and randomly changing elements in the second array. For both mutation and initialization, 0 is three times more likely than 1 — as that bias helps prevent the knapsack from being over capacity. For cross-over, a starting and ending point are first randomly selected in the first parent’s tour. This section is copied directly into the child. Then the rest of the cities are added in the order they appear from the second parent (the inverse is done for the second child). For the items, a cross over point is randomly selected and the first part of one parent is concatenated with the second part of the other parent.

Fitness (F)

Fitness is set at $Z(\Pi, P)$.

Diversity (D) For diversity we wanted to measure the trade-off between the two subproblems (Traveling Salesman and Knapsack). To measure this we estimated the optimal solution to each subproblem using single population genetic algorithms that optimized just the subproblems. For each of the 6 instances we ran a genetic algorithm for 1,000 generations with a population size of 300 to estimate the minimum travel distance (*MinTravel*), maximum travel distance (*MaxTravel*), and maximum profit (*MaxProfit*) from collecting items. We used these values to estimate how close to being optimal each subproblem of a TTP solution is.

For a solution with total distance traveled t , the distance value is measured as $T = \frac{MaxTravel - t}{MaxTravel - MinTravel}$.

For a solution with total profit p , the profit value is measure as $P = \frac{p}{MaxProfit}$.

These two values are then combined into one diversity function that represents the trade-off between profit and distance, with lower values representing solutions prioritizing maximum profit (like the Knapsack) and higher values prioritizing minimum distance (like the Traveling Salesman).

$$D = \frac{P}{P + T}$$

This function is used to place each solution into one of 10 bins.

Static Constraints (S) The only static constraint for this problem space is that the ending weight (w_{x_n}) must be under the knapsack capacity (W). However, to provide additional support, we add the additional constraints that $w_{x_n} \leq W * i$ for $i \in [1, 4]$ with a step size of 0.2.

Variable Constraints (V)

As a variable constraint a user can say that an item i should be collected on the tour. The variable constraints are considered contradictory if more than one constraint exists for a single item (which would be redundant), or if the weight of all items that need to be collected exceeds the capacity of the knapsack.

7.2 Personas

To simulate users (or other variable environments), algorithms will be tested using with 4 procedural personas. Personas are prompted every 50 generations, at which point they are given the current feasible population and can choose to either add or remove a constraint.

- **Exploratory Persona:** this persona randomly adds or removes a constraint at each generation every cycle. This is meant to simulate users or situations with no clear pattern.
- **Strict Persona:** this persona randomly adds a non-contradictory constraint at each cycle. If a non-contradictory constraint cannot be found it does nothing. This is meant to test algorithms in the most difficult situations.
- **Adaptable Persona:** this persona adds a random constraint that is already satisfied in the current population. This is meant to simulate users who look for patterns or valuable individuals in the existing population, along with testing for how well the algorithms can exploit existing solutions.
- **Cycle Persona:** this algorithm adds a random constraint for the first two cycles, and then removes the first constraint added. It then switches between adding a new constraint and removing the oldest constraint (such that there are between 1 and 2 constraints). This provides a constantly changing environment, while still giving the algorithm 2-3 cycles to learn a constraint.

7.3 Baseline Algorithms

As inspiration and base of comparison, there are 4 provided algorithms in the competition code-base.

7.3.1 Random Restarts

This algorithm maintains two populations. The infeasible population contains individuals that violate at least one static or variable constraint. Infeasible individuals are selected for reproduction based on how many constraints they violate. The feasible population satisfies all constraints and is organized into bins based on the diversity function. Each bin maintains the top N (determined by the maximum memory capacity) individuals sorted by fitness. If there is no room left in a bin, an individual is only added if it is at least as good as the worst individual in the bin (and the worst individual is removed). Feasible individuals are selected by first randomly choosing a bin (of those with at least one individual in it), and then selected within a bin based on fitness. When the variable constraint list is changed, the population (both feasible and infeasible) is dumped and re-initialized with random individuals.

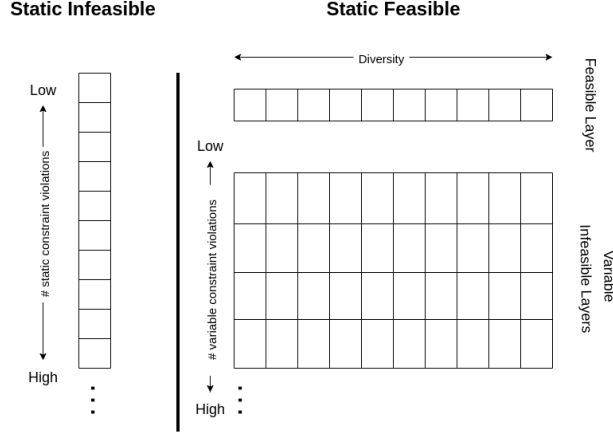


Figure 2: Diagram showing the populations for the VC-MAP-Elites algorithm. Note that each cell in the static feasible grid contains multiple individuals, while every element in the static infeasible array contains only one individual.

7.3.2 Shuffling

This algorithm maintains two populations, like in random restarts. However, when the variable constraint list is changed, the old population is not dumped. Instead the existing population is re-sorted into the infeasible population and feasible bins.

7.3.3 Filtering Algorithm

This algorithm similarly maintains two populations. However, an individual is placed in the infeasible population only if it violates the *static* constraints, and *variable* constraints are ignored. Any individual that violates a variable constraint is filtered out (but remains in the population) just before being returned.

7.3.4 Variable Constraint MAP-Elites (VC-MAP-Elites)

This algorithm takes inspiration from the MAP-Elites algorithm and applies it to the landscape of variable constraints. The pseudo code for this algorithm is provided in Algorithms 1, 2 and 3.

This algorithm keeps track of two populations: the static infeasible population, and the static feasible population. This is shown in Figure 2.

The static infeasible population contains individuals that violate one or more static constraints (S). Individuals in the static infeasible population are stored in an array and are selected based on the number of static constraints they do satisfy.

The static feasible population contains individuals that satisfy all static constraints (S), but may violate one or more variable constraints (V). This popula-

Algorithm 1 VC-MAP-Elites Selection

```
if True at rate  $num - feasible * 2 / total - population$  then
    set  $Row = select([h, h - 1, ..., 0])$ 
    set  $Bin = \text{random non-empty bin in } Row$ 
    set  $Ind = select([i_{fit} \text{ for } i \in Bin])$ 
    Return  $Ind$ 
else
    set  $i_{sat} = \text{number } c \in S \text{ satisfied } \forall i \in StaticInfeasible$ 
    set  $Ind = select([i_{sat} \text{ for } i \in StaticInfeasible])$ 
    Return  $Ind$ 
end if
```

tion is represented as a two dimensional grid. The x-axis stores individuals that vary based on the diversity function (D). The y-axis, with height of h (by default $h = 4$), stores individuals based on the percentage of variable constraints they satisfy. The 0th row contains individuals that satisfy all variable constraints. The remaining $h - 1$ rows are assigned intervals of percent of variable constraints satisfied — with larger row numbers having more variable constraint violations. When selecting an individual, the row is selected first, with probability inversely proportional to the row number (large row numbers are selected less often).

VC-MAP-Elites uses a dynamic memory system. The total memory capacity is first allocated between feasible and infeasible individuals (with a default 50% – 50% allocation). The feasible memory is spread between the 0th row of the static feasible grid, and the infeasible memory is spread between the static infeasible population and the remaining $h - 1$ rows of the static feasible grid. Feasible individuals are placed in the grid until the memory capacity is exceeded, and the individual with the lowest fitness is removed (regardless of diversity bin). Similarly, infeasible individuals are added to the static infeasible or static feasible populations until the memory capacity is exceeded. If there are individuals in the static infeasible population, they are removed first in the order of individuals that violate the most constraints. When the static infeasible population is empty, individuals are removed from the static feasible grid — with the individuals with lowest fitness being removed first.

References

- Ahmed Khalifa, Roberto Gallotta, Matthew Barthet, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. 2025. The Procedural Content Generation Benchmark: An Open-source Testbed for Generative Challenges in Games. In *Proceedings of the 20th International Conference on the Foundations of Digital Games*. 1–12.
- Adel Nikfarjam, Aneta Neumann, and Frank Neumann. 2024. On the use of

Algorithm 2 VC-MAP-Elites Place (Ind)

```
if  $Ind$  satisfies all  $c \in S$  then
    set  $p$  = percent of  $c \in V$  violated
    if  $p$  is 0 then
        set  $l = 0$ 
    else
        set  $l = \lfloor p * (h - 2) \rfloor + 1$ 
    end if
    Add  $Ind$  to  $StaticFeasible[l][D(Ind)]$ 
else
    Add  $Ind$  to  $StaticInfeasible$ 
end if
if Number of Feasible is exceeded then
    Remove  $i \in StaticFeasible[0]$  with lowest fitness
end if
if Number of Infeasible is exceeded then
    if Length  $StaticInfeasible$  is greater than 0 then
        Remove  $i \in StaticInfeasible$  that violates the most  $c \in S$ 
    else
        Remove  $i \in StaticFeasible[1 : h - 1]$  with lowest fitness
    end if
end if
```

Algorithm 3 Variable Constraint MAP-Elites (One Generation)

```
if  $V$  has changed then
    Re-Organize Static Feasible Grid
end if
for Children in population size do
     $p1, p2 = VC\text{-}MAP\text{-}Elites\ Selection()$  X 2
     $c1, c2 = X(p1, p2)$ 
     $c1, c2 = M(c1), M(c2)$ 
    VC-MAP-Elites Place( $c1$ ), VC-MAP-Elites Place( $c2$ )
end for
```

quality diversity algorithms for the travelling thief problem. *ACM Transactions on Evolutionary Learning and Optimization* 4, 2 (2024), 1–22.

Sergey Polyakovskiy, Mohammad Reza Bonyadi, Markus Wagner, Zbigniew Michalewicz, and Frank Neumann. 2014. A comprehensive benchmark set and heuristics for the traveling thief problem. In *Proceedings of the 2014 annual conference on genetic and evolutionary computation*. 477–484.

Fiona Shyne, Kaylah Facey, and Seth Cooper. 2024. Procedurally puzzling: on algorithmic difficulty and player experience in QD-generated logic grid

puzzles. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 20. 127–137.

Fiona Shyne, Kaylah Facey, and Seth Cooper. 2025. Growing a Puzzle Garden: Exploring Casual and Serious Features in a Mixed-Initiative Logic Puzzle Authoring Tool. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 21. 326–336.